阳 光 学 院

本 科 毕 业 论 文（设 计）

题　　目：基于蜜罐技术的网络威胁捕获系统设计与实现

院（系）别：人工智能学院

专　　业：网络工程（专升本）

年　　级：2024

学　　号：2024189139

姓　　名：陈威宏

指导教师：黄振祥(高级工程师)、林翔(助教)

2026 年 05 月

阳光学院本科生毕业论文（设计）诚信承诺书

毕业论文（设计）题目		基于蜜罐技术的网络威胁捕获系统设计与实现			
学生姓名	陈威宏	年　级	2024 级	学　　号	2024189139
所在学院	人工智能学院			所学专业	网络工程（专升本）

学生承诺

我承诺在毕业论文（设计）活动中遵守学院有关规定，恪守学术规范，在本人的毕业论文（设计）中未剽窃、抄袭他人的学术观点、思想和成果，未篡改实验数据，如有违规行为发生我愿承担一切责任，接受学院的处理。

学生（签名）：

年　　月　　日

指导教师承诺

我承诺在指导学生论文（设计）活动中遵守学院有关规定，恪守学术规范，经过本人认真的核查，该同学的毕业论文（设计）中未发现有剽窃、抄袭他人的学术观点、思想和成果的现象，未发现篡改实验数据。

指导教师（签名）：

年　　月　　日

基于蜜罐技术的网络威胁捕获系统设计与实现

摘要

针对网络安全实验环境中未知威胁发现不及时、攻击行为难以持续留痕的问题，本文设计并实现了一套基于蜜罐技术的网络威胁捕获系统。系统由 Web 诱捕端、SSH 仿真端和管理端组成，处理链路覆盖攻击接入、行为判定、统一存储和可视化联查等环节。结合实验场景下的安全边界与部署条件，系统采用中交互蜜罐策略，在保证交互真实性的同时控制宿主机风险，以便更稳定地记录攻击过程并积累可分析样本。

实现过程中，本文重点解决了两个问题：一是将 SSH 登录结果、命令审计记录和 Web 请求行为整理为同一事件结构，在事件来源、事件类型、来源 IP、会话标识等公共字段上统一治理，减少跨协议联查时的数据清洗成本；二是引入“多特征加权 + 双阈值分层”的风险判定机制，将展示链路与告警链路分离，使路径探测、注入语义、命令执行等信号能够按风险强弱逐步累积，在保证威胁捕获能力的同时尽量降低低风险噪声干扰。具体实现上，SSH 端基于 Paramiko 完成握手、认证与命令交互仿真，Web 端基于 Flask 对敏感路径探测、异常参数和自动化扫描行为进行捕获与评分，最终由管理端统一展示事件分布、来源统计和会话轨迹。

系统接入外网连续运行后，累计记录事件 85082 条，其中 SSH 事件 84273 条、Web 事件 809 条。在“外网流量接入 + 管理端回查”的测试条件下，Web 注入专项回放的捕获率为 82.0%，管理端核心接口压测成功率为 100.0%。综合运行与测试结果表明，该系统能够较稳定地完成攻击行为留痕、事件汇聚与可视化联查，适用于网络安全实验和研究场景下的威胁观察与研判分析。

关键词： 蜜罐技术　网络威胁捕获　统一事件模型　协议仿真　风险判定

目　录

1 绪论 1
1.1 开发背景及意义 1
1.2 国内外研究现状 1
1.3 研究内容 2
1.4 论文的主要结构 2
2 相关开发技术 4
2.1 蜜罐技术原理 4
2.2 Python 与 Flask 框架 4
2.3 SSH 协议与 Paramiko 库 4
2.4 SQLite 与 MySQL 数据库技术 4
2.5 ECharts 可视化技术 5
2.6 统一事件模型与数据治理技术 5
2.7 本章小结 5
3 系统分析 6
3.1 可行性分析 6
3.2 功能需求分析 6
3.3 非功能需求分析 8
3.4 本章小结 8
4 系统设计 9
4.1 功能流程设计 9
4.2 功能结构设计 10
4.3 系统界面设计 13
4.4 数据库设计 15
4.5 本章小结 21
5 系统实现 22
5.1 系统环境配置 22
5.2 SSH 协议仿真功能实现 22
5.3 Web 漏洞诱捕功能实现 23
5.4 攻击日志采集与解析实现 25
5.5 态势感知大屏展示实现 26
5.6 本章小结 28
6 系统测试 29

6.1 系统测试目的 29
6.2 系统测试过程 29
6.3 系统测试结论 32
6.4 本章小结 32
结论 33
参考文献 34
致谢 35

1 绪论

1.1 开发背景及意义

1.1.1 网络安全形势

国家已就关键信息基础设施安全保护建立专门制度[1]。公开通报显示，面向公网的 Web 攻击和命令执行攻击一直较为活跃[2]。在实际部署中，不少业务系统直接部署在云服务器上并对外提供访问，页面入口、远程管理端口和接口路径长期处于可达状态，往往最先遭到扫描和试探。

随着业务在线化程度提高，面向公网的攻击探测和持续试探更为常见[2]。在这种背景下，完全依赖特征库更新的被动防护往往难以及时覆盖变形攻击[3,4]。尤其在 Web 侧，注入、认证与访问控制相关问题仍是高频风险来源，防护策略需要持续迭代[3]。

1.1.2 课题研究目的及意义

针对被动防护对未知威胁感知滞后的问题，本课题实现了一套可持续运行的蜜罐诱捕系统。蜜罐技术属于主动防御思路，其核心不在于阻断全部访问，而在于将攻击行为引入可观测环境。与防火墙、IDS 的规则拦截逻辑相比，蜜罐更强调行为证据采集与过程留痕。

本课题的研究目的，在于构建面向外网攻击行为的持续观测链路，使探测、爆破、命令执行和参数攻击等过程能够被稳定采集、统一存储，并支持后续联查与回溯。

其研究意义主要体现在两个方面：一是将 SSH 与 Web 两类异构行为纳入同一治理口径，为跨协议统计、会话追踪和来源聚合提供基础；二是在受控交互环境中保留较完整的行为证据，为威胁研判、规则优化和样本复核提供依据。

1.2 国内外研究现状

1.2.1 国外研究现状

国外在 Web 安全治理方面已形成较成熟的识别与处置思路。OWASP Top 10 等行业实践持续更新高风险问题清单与治理重点，也为 Web 安全防护和测试提供了常用参考[3,4]。在蜜罐研究方面，早期虚拟蜜罐框架为后续工程化演进打下了基础[5]。从现有开源方案来看，单协议诱捕仍占主流。例如 Cowrie 主要聚焦 SSH 或 Telnet 仿真，Glastopf 主要面向 Web 漏洞诱捕[6,7]。这类工具在各自场景中已经比较成熟，但事件字段、标签口径和接口输出并不统一，跨协议联合分析时往往还要额外做数据清洗与映射。对于实验与研究场景而言，这一步既增加了部署复杂度，也拉高了后续联查成本。

1.2.2 国内研究现状

近几年，国内已出现面向蜜罐部署与诱捕应用的研究和工程实践[8,9]。相关工作围绕登录诱捕、终端仿真和内网防护等方向展开，说明国内在策略设计、行为留痕和可控部署方面已有探索。从已公开成果来看，现有工作更多聚焦单点能力优化，例如登录诱捕、终端仿真或局部规则识别，对跨协议统一建模、轻量化部署和可视化联动的综合讨论仍然不多。

总体来看，国内相关研究已在登录诱捕、终端仿真和局部防护等方面形成一定基础，但对跨协议统一建模、分层风险处置以及轻量化部署协同的系统化讨论仍然较少，这也构成了本文进一步展开的切入点。

1.3 研究内容

本文研究对象是一套由 Web 诱捕端、SSH 仿真端和管理端组成的网络威胁捕获系统，运行链路覆盖接入采集、风险识别、统一入库和可视化联查。全文重点讨论两项工作：一是将 SSH 会话行为与 Web 请求行为统一映射为事件结构，用于支撑跨协议统计与联查；二是对路径、请求头、参数和请求体中的风险信号进行加权累计，并以双阈值区分展示级可疑事件与高危告警。

其中，统一事件模型解决的是“不同协议的行为日志如何放到一起看”的问题。SSH 侧重点记录用户名口令、认证结果和命令行为，Web 侧重点记录请求方法、路径、参数和请求体，两者原始结构差异较大，若直接沿用各自日志格式，管理端很难在同一界面完成统计、回溯和联查。为此，本文从后续研判需求倒推公共字段，把需要统一统计和检索的信息保留在事件层，把协议特有且需要继续留证的内容保留在原始请求表或扩展字段中。另一项工作关注的是如何区分需要持续观察的异常行为与需要及时提示的高风险行为。系统不把单次命中直接等同于攻击成立，而是让路径、请求头、参数和请求体中的多种信号先进入累计，再以双阈值把可疑事件和高危告警分开处理，减少低风险噪声对告警链路的挤占。

课题推进按照“需求收敛—方案实现—结果验证”的顺序展开。前期先根据文献与开源方案梳理可复用方案，再统一需求约束、模块边界和数据口径；进入实现阶段后，SSH 诱捕、Web 诱捕与管理端按模块联调，最后结合功能用例、接口压测和公网连续运行样本对结果进行交叉核验。

1.4 论文的主要结构

全文主体按照“问题提出—方案设计—工程实现—测试验证”的主线展开：第一章说明研究背景与现状，第二章给出关键技术基础，第三章与第四章分别对应需求分析和系统

设计，第五章描述核心实现细节，第六章给出功能、性能与真实运行结果；结论部分再对全文工作进行归纳，并说明后续优化方向。

2 相关开发技术

2.1 蜜罐技术原理

蜜罐的核心是把可疑访问引到可观测区域并保持边界可控，其重点在于行为采集和过程留痕，而非承载真实业务[5,10]。工程上通常按交互深度分为三类。

低交互蜜罐通常只模拟端口开放状态和基础协议响应，不提供真实业务执行面。该类型安全边界清晰、并发承载能力较好，但交互深度不足，容易在深入探测时被识别为诱饵环境。[5]

高交互蜜罐直接提供真实系统或应用环境，数据最完整，但隔离、防横向移动和运维监控压力也最高。隔离不足时，蜜罐可能被当作跳板，部署与维护成本会明显上升[5]。

中交互蜜罐开放有限业务行为面，例如登录会话、目录浏览、命令回显。攻击者可以完成一定程度的交互，但执行路径仍受限制，因此在可控风险前提下能够产出更高价值行为数据[5]。结合本课题的部署条件与安全边界，系统最终选用中交互方案，以兼顾样本完整性、运行风险和维护成本。

2.2 Python 与 Flask 框架

Python 标准库提供套接字、线程和字节流处理能力，可支撑 SSH 与 Web 诱捕模块开发[11]。结合现有工程生态，协议处理、并发连接管理和日志治理较易落地。

Flask 采用轻量化设计，适合快速搭建请求处理与接口服务[12]。系统将审计、评分与证据落库统一纳入请求处理链路，以兼顾开发效率与后续维护。

2.3 SSH 协议与 Paramiko 库

SSH 仿真依托 Paramiko 服务端能力实现。系统扩展 ServerInterface 接管认证与通道流程，在应用层记录登录尝试、会话标识和命令交互，并按统一事件模型入库[13]。

认证过程不再只输出“成功/失败”二值结果，而是转化为可观测事件，并与后续会话交互串联，提升证据链完整度。

2.4 SQLite 与 MySQL 数据库技术

实验阶段选用 SQLite 做事件持久化。SQLite 采用嵌入式、单文件、无独立服务进程的形态，便于快速联调[14]；在此基础上，系统围绕高频查询字段补充索引设计，以支撑

日志写入、联查检索和统计接口响应。需要承接更高并发或远程访问时，部署侧可通过配置将存储后端切换到 MySQL，调整重点主要在连接方式、库表初始化和运行参数上。由于诱捕流程与评分规则不因数据库变化而重写，改动主要集中在数据访问层。

2.5 ECharts 可视化技术

可视化层以 ECharts 组织图表展示[15]。结合来源地理信息与时间窗口统计，管理端可以集中查看威胁分布、类型变化和来源聚集趋势。

2.6 统一事件模型与数据治理技术

为解决多源日志口径不一致的问题，系统在数据层引入统一事件模型。该模型以事件时间、来源类型、风险等级、标签和摘要为基础字段，把 SSH 与 Web 行为整理到同一结构之下，再用标准化字段支撑跨端统计、关联检索和会话联查，从而减少联查前的数据清洗与字段映射工作。

数据进入治理环节后不会直接原样入库。系统先对敏感参数进行替换，对超长载荷按阈值截断，并将补充说明写入扩展字段，以在保留关键取证信息的同时控制存储体积和后续检索压力。

2.7 本章小结

本章所述技术分别对应系统实现中的关键环节：协议仿真负责接入并记录攻击行为，Web 请求处理负责识别入口流量，数据库持久化负责保存证据数据，可视化渲染负责展示分析结果，统一事件模型则用于将这些结果整理为同一套数据结构，供后续联查、统计和展示使用。

3 系统分析

3.1 可行性分析

3.1.1 技术可行性分析

从实现条件看，本项目所用 Python、Flask、Paramiko、SQLite/MySQL 均为成熟技术栈，资料完整、版本稳定。SSH 仿真、Web 请求采集、事件入库和可视化展示都有可复用的工程路径，技术风险主要集中在并发连接与日志写入冲突，已通过连接限额、重试和索引策略进行控制。

3.1.2 经济可行性分析

从投入看，系统可运行在常规个人电脑上，无需额外采购专用硬件。核心组件采用开源方案，软件采购成本接近零。配合统一启动入口和模块化结构，部署、复现实验、后续维护都在毕业设计可承受范围内。

3.1.3 社会可行性分析

在网络安全实验与研究场景中，蜜罐系统可以在可控边界内复现真实攻击行为，便于开展攻防复盘和行为分析。系统不承载生产业务，并设置了清晰交互边界，能够满足安全隔离、可追溯和可管理方面的要求，适合用于实验与研究场景。

3.1.4 运行可行性分析

系统采用统一入口和分模块部署方式，启动、联调与停机流程明确，便于在实验环境中反复验证。各模块职责边界明确，出现异常后可快速定位到接入层、治理层或展示层，运维成本处于可控范围。

3.2 功能需求分析

3.2.1 需求概述

系统由两类角色驱动。攻击侧持续产生可观测行为，管理侧负责联查与研判。功能要求覆盖协议诱捕接入、行为采集与评分、统一事件治理和可视化追溯分析。

结合交互真实性、采集深度与宿主机安全边界三方面要求，系统在需求阶段即确定采用中交互方案，在保留必要行为样本的同时，不向攻击侧暴露可直接作用于宿主机的真实

执行面。需求上要求系统能够连续保留 SSH 与 Web 两类入口的关键行为证据，并将攻击活动限制在受控边界内，以兼顾观测深度与运行安全。

为保证后续统计口径一致，需求阶段先统一字段命名、风险分级与展示规则。展示层统一读取标准化事件数据，不直接拼接原始日志文本，以减少语义混杂和展示干扰。

从业务目标看，系统需要提升可疑行为可观测性，统一多源日志统计口径，并保证展示结果能够回溯到具体事件记录。在需求定义阶段，既要保证功能流程连贯，也要保持模块独立，避免后续扩展产生高耦合。

3.2.2 用例分析

用例建模按“角色 - 行为 - 数据流”三条线并行展开。外部参与者分为攻击者与管理员，内部处理对象分为协议接入、行为分析、事件持久化和展示模块，并在同一模型内明确输入、处理和输出边界。经过这种拆分，复杂流程可以进一步分解为若干职责清晰的处理环节，便于后续实现、联调和测试用例设计。系统用例图如图 3-1 所示。

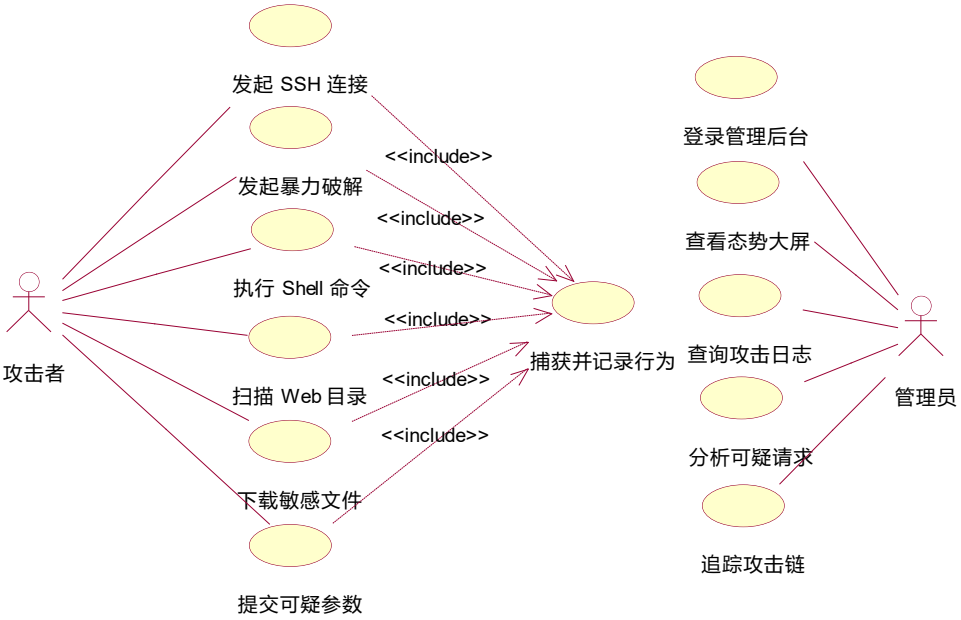


图 3-1 系统用例图

攻击者侧围绕“探测 - 试探 - 留痕”组织核心交互，典型动作包括 SSH 连接尝试、口令爆破、命令输入、目录扫描和可疑参数提交。在用例图中，“捕获并记录行为”被设置为公共子用例，SSH 连接尝试、命令输入、目录扫描和参数探测等动作均通过包含关系接入这一用例。采用这一设计后，不同入口可以共用同一条采集与落库路径，既减少重复建模，也保证证据链完整；后续若补充新的攻击场景，只需沿现有关系继续扩展即可。

管理员侧围绕“查看 - 联查 - 研判”展开，具体包括登录后台、查看态势大屏、查询日志、分析可疑请求和追踪攻击链。所有管理动作均以认证通过为前置条件，输入数据

来自攻击侧持续落库的结构化事件，输出结果服务于定位、追溯和处置决策，整体数据流向与后文类图、时序图保持一致。

该模型同时明确了各对象的输入输出职责：接入对象负责解析和预检，分析对象负责特征提取与评分，事件对象负责封装存储，展示对象负责读取渲染。该划分既能保持诱捕侧与展示侧职责分离，又能保证数据链路前后衔接；后续替换局部策略或展示组件时，也无需重构整体架构。

3.2.3 用例描述

从模型关系看，攻击侧操作主要落在 SSH 与 Web 两类入口上。SSH 入口对应连接尝试、口令爆破以及认证后的命令执行，Web 入口对应目录扫描、敏感文件探测和异常参数提交。二者虽然表现形式不同，但都会进入同一条记录链路：SSH 侧先写入认证结果，再挂接后续会话与命令输入；Web 侧在接收到路径访问或参数提交后进入请求分析与风险评分流程。最终生成的结果统一进入事件记录链路，供日志联查和攻击链追踪使用。

管理员侧用例包括登录管理后台、查看态势大屏、查询攻击日志、分析可疑请求和追踪攻击链，其业务输入均来源于攻击侧产生的结构化事件数据。该角色的重点是借助统一事件模型完成定位、联查与溯源分析。

3.3 非功能需求分析

非功能需求重点落在性能、安全性、稳定性和可维护性，并统一受“数据留痕、过程可追溯”约束。

性能方面，管理端核心接口需保持秒级响应，事件写入链路在持续流量下不能出现明显积压。安全方面，诱捕侧与管理侧保持边界隔离，入库数据按字段类型执行差异化整理，对令牌、密钥等高敏内容进行替换，并对长文本做长度控制。稳定性方面，系统需支持连续运行和异常恢复，避免长时间阻塞。维护性方面，统一配置、日志结构与接口定义，便于定位问题和后续扩展。

除性能与安全要求外，展示层还需保持统计口径一致。核心统计、会话分析与请求轨迹统一使用同一时间窗口，避免跨模块冲突。运行管理参照网络安全事件报告要求[16]，保证日志留存、事件分级和处置记录可追踪。

3.4 本章小结

本章完成可行性、功能需求与非功能需求分析，明确了角色边界、业务目标和质量约束，为后续系统设计与实现提供了需求基线。

4 系统设计

4.1 功能流程设计

按数据流拆解，系统主流程可分为四个环节，即协议接入、行为捕获、事件治理与管理展示。SSH 流量进入仿真端后，先完成握手、认证与命令审计，并直接形成统一事件；Web 流量进入诱捕端后，先经过请求预检和原始请求留存，再执行风险评分，并在达到阈值后升级写入统一事件。两条链路最终由管理端在统一时间窗口内完成统计、联查与追踪展示。

该流程重点控制两方面内容：一是持续向攻击侧提供反馈，保证行为样本持续产生；二是收紧运行边界，避免诱捕面反向影响宿主环境。管理侧在统一时间窗口内观察事件演进，以保证统计、联查和追踪结果保持一致。

协议接入、行为分析、事件持久化与展示对象之间的静态关系在类图中展开，如图 4-1 所示。

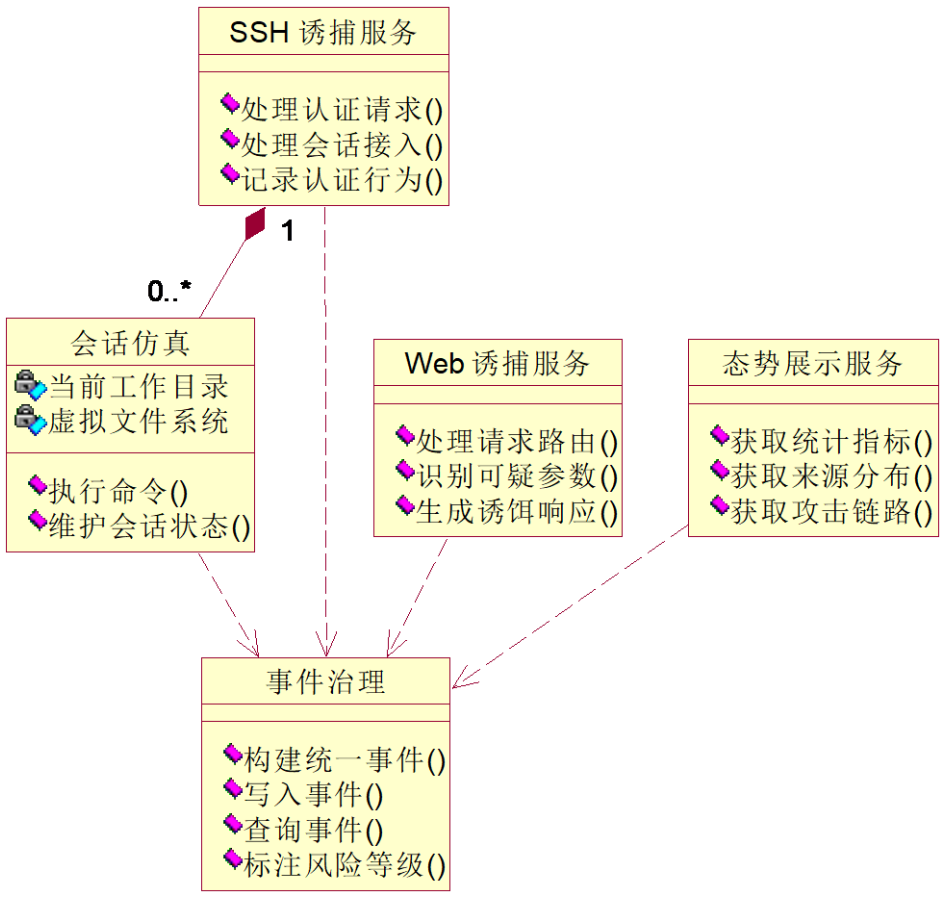


图 4-1 系统类图

系统并非将 SSH、Web 和管理端作并列设置，而是先分入口接入，再在持久化阶段汇聚到统一事件、原始请求和会话追踪三类数据结构，最后由管理端集中展示。采用这种组织方式后，管理端既可以按协议分别观察，也可以在同一时间窗口内把认证失败、命令审计、路径探测和注入尝试纳入同一条分析链路。

4.2 功能结构设计

功能结构按三级展开。顶层是网络威胁捕获系统，中层是诱捕接入、事件治理、态势展示，底层再细化为协议接入、特征解析、统一落库、地理富化、链路追踪。分层后职责更清楚，后续扩展新协议或新分析模块时改动范围可控。

模块调用关系遵循单向流转原则。接入层负责采集行为，治理层负责统一处理，展示层负责统计与可视化，不出现反向依赖。

系统从整体到核心模块、再到各子模块的逐级分解关系如图 4-2 所示。

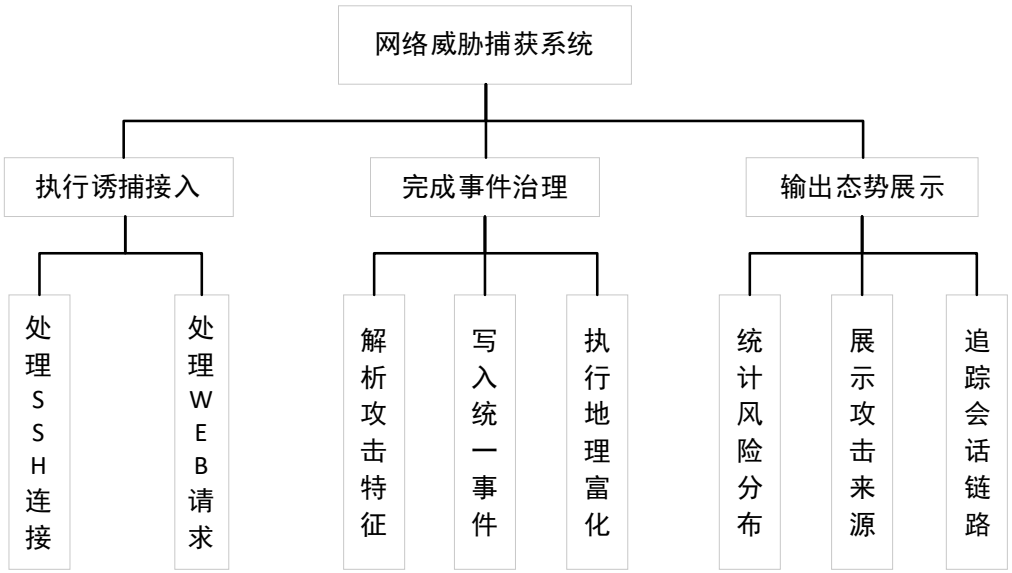


图 4-2 系统功能结构图

按功能层次继续向下拆开后，诱捕接入、事件治理和态势展示三部分的职责更加明确。顶层对应课题整体目标，中层把三类核心职责分开，底层再落实到协议接入、特征解析、统一落库、地理富化和链路追踪等具体环节。其中，统一事件模型主要落在事件治理层，用于把 SSH 与 Web 的异构记录整理为同一口径；风险判定与特征解析位于接入层和治理层之间，负责把原始行为转化为后续可统计、可联查的结构化结果。在此结构下，后续新增协议入口或调整评分规则时，不必连带改动整个展示链路。

管理员的实际操作通常从登录后台开始，接着查看大屏、筛选日志、联查请求，最后再决定是否继续处置。权限校验是这条路径的入口，完整流程如图 4-3 所示。

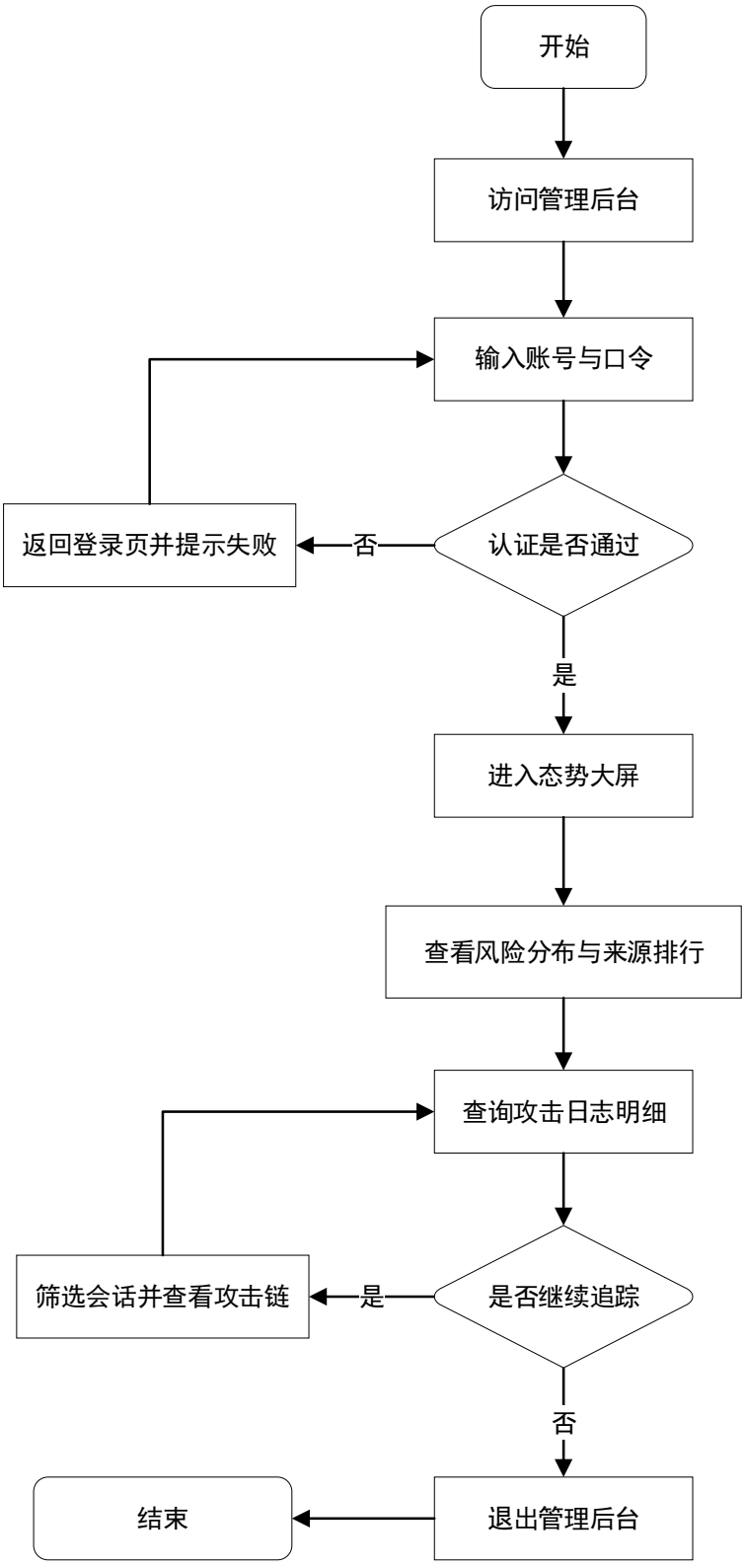


图 4-3 系统操作流程图

管理端的使用过程并非在多个页面之间无序切换，而是沿着“先确认身份、再看总体、后做联查”的顺序推进。登录步骤负责收紧访问边界，大屏先给出当前时间窗口下的整体

态势，日志筛选和请求联查再把宏观统计压到具体行为记录。由此，管理员既能先把握整体趋势，也能随时回到单条事件核对证据。

SSH 仿真模块在握手和认证阶段接管协议流程并持续记录攻击行为。账号校验通过后即初始化内存虚拟环境，再由仿真终端执行指令交互，时序关系如图 4-4 所示。

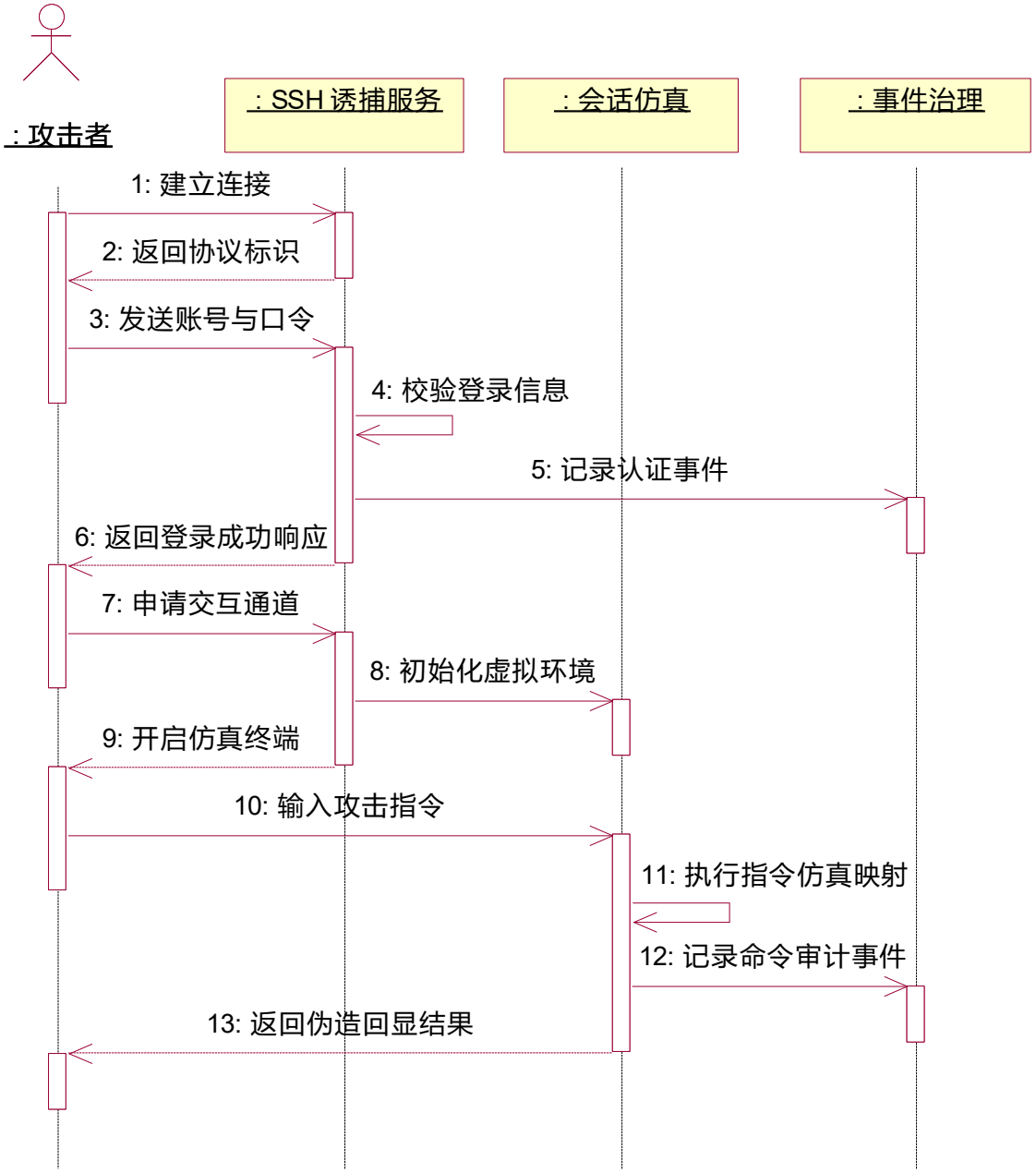


图 4-4 SSH 交互时序图

SSH 侧的关键在于，从连接建立开始即沿同一连接标识持续写入，而不是在认证通过后再另起一套记录逻辑。前半段负责握手、口令校验和认证结果记录，后半段负责终端初始化、命令接收、仿真执行与回显。据此回看时，“谁在尝试登录”“是否登录成功”“登录后执行了什么命令”都能落回同一条会话轨迹，而不是拆成彼此孤立的几段日志。

Web 诱捕模块通过接管特定业务路由实时分析 HTTP 请求特征。识别到风险后，系统先记录行为特征，再返回诱饵响应，在不暴露真实后台的前提下完成捕获，时序关系如图 4-5 所示。

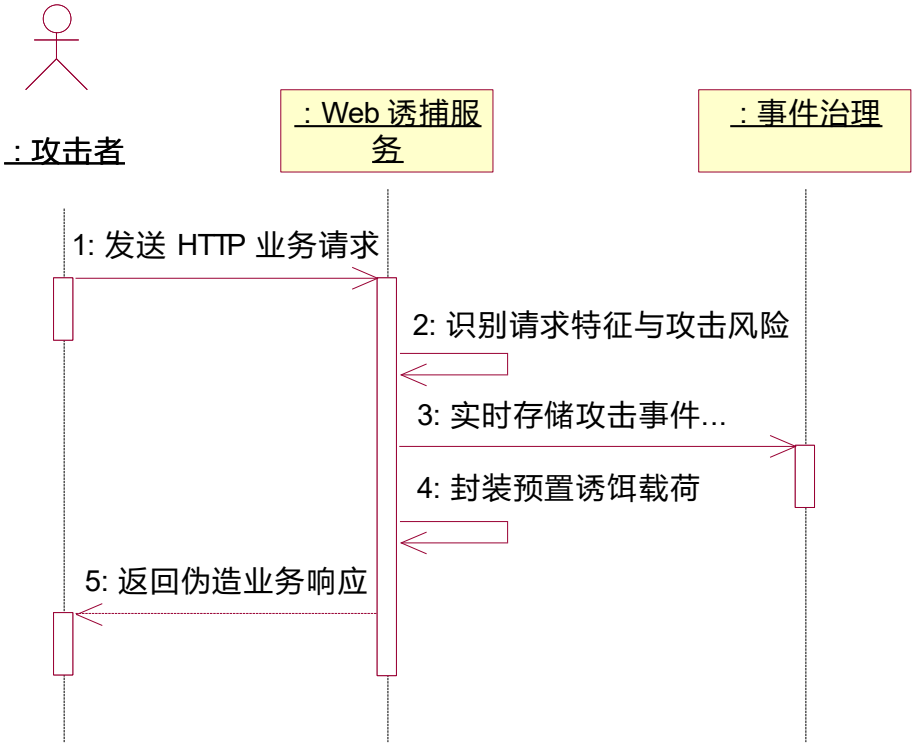


图 4-5 Web 诱捕时序图

Web 侧把处理顺序固定为“先留证、再判定、后响应”。请求到达后先经过入口约束和基础审计，再把方法、路径、参数、请求头和请求体写入原始请求记录，随后才进入风险评分和事件升级流程；命中高风险入口或可疑特征后，系统不会直接暴露真实后台状态，而是返回伪装响应。由此，请求记录在响应返回前已经保留下来，攻击者从页面表现上仍会认为目标存在可继续试探的空间。

4.3 系统界面设计

界面设计采用原型先行策略，先验证布局和交互，再进入真实数据联调。攻击侧界面保持终端与后台页面风格，管理侧采用大屏分区布局，用于拉开信息层级并减轻阅读负担。

控件层使用按钮、输入框、下拉框、标签卡片和表格容器，覆盖检索刷新、条件筛选、时间窗口切换、来源类型切换和事件列表展示等核心操作。本阶段只验证可用性与可读性，不展开代码实现。

SSH 终端原型重点体现命令输入区、回显区和提示符样式。SSH 蜜罐伪装终端界面原型图如图 4-6 所示。

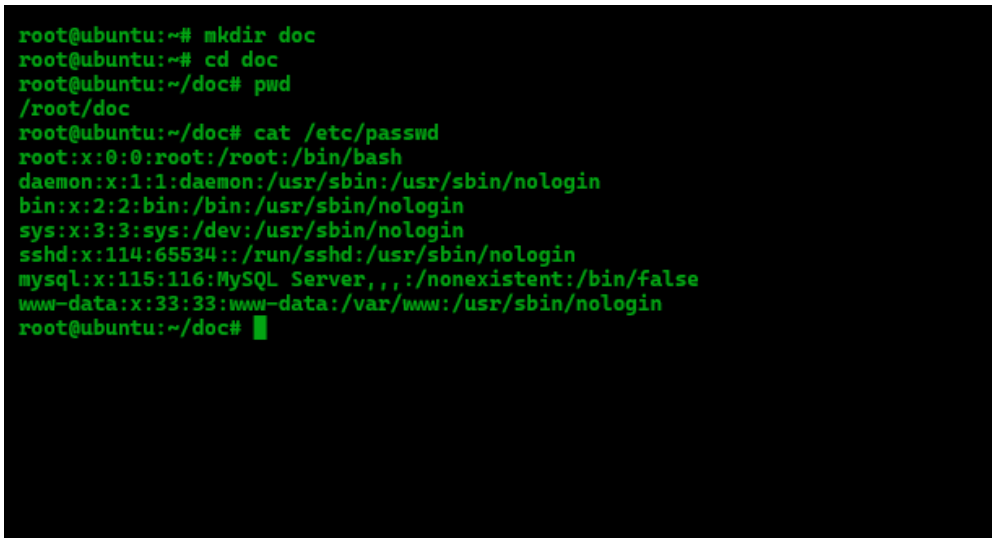


图 4-6 SSH 蜜罐伪装终端界面原型图

中交互 SSH 界面中影响攻击者是否继续停留的因素，主要体现在提示符样式、输入区位置和命令回显区域。界面保留类 Ubuntu 终端风格，目的不在于完整复制真实系统，而在于让常见探测与信息收集命令能够持续得到反馈，从而延长交互过程并提高命令样本的留存概率。

Web 后台原型突出登录入口、导航结构和内容区域分层。Web 诱捕后台界面原型图如图 4-7 所示。



图 4-7 Web 诱捕后台界面原型图

后台原型突出页面骨架的可信度。界面保留登录入口、左侧导航和主内容区等常见后

台结构，使攻击者在访问后台路径时能够看到熟悉的页面组织方式，进而继续尝试登录、切换栏目或访问子路径。对本系统而言，其作用不在于承载真实生产业务，而在于提供受控的后台交互场景，为后续登录尝试、路径探测、参数提交和任务操作行为留存更完整的样本。

态势感知大屏原型设计先确认图表区域划分、信息密度和阅读顺序是否合适，视觉效果放在后续再调整，以避免接入真实数据后反复修改布局。其界面原型如图 4-8 所示。



图 4-8 态势感知大屏界面原型图

原型阶段先确定统计总览、地理来源、事件明细和联查区域之间的位置关系：统计总览对应整体态势判断，地理来源用于识别攻击分布，事件明细承担单条取证，联查区域负责沿来源 IP、原始请求和会话标识继续回溯。重点不在于追求视觉细节，而在于先稳定阅读顺序。在接入真实数据后，即使事件总量上升或日志明细变长，页面也不会轻易打乱“先看整体、再看来源、最后回到单条行为”的观察节奏。

4.4 数据库设计

数据库设计是系统持久化链路的核心。为保证多源异构攻击行为既能准确落库又能高效检索，本文按照“概念设计 - 逻辑设计 - 物理设计”的顺序展开。

4.4.1 概念结构设计

概念结构设计先从联查、回溯和证据复核需要重点保留的信息出发。数据库层面保留四类核心实体：攻击源、事件、会话和原始请求。攻击源描述行为来源及其地理属性，事

件承载认证记录、命令审计和 Web 风险判定结果，会话用于串联同一连接或同一来源下的连续操作，原始请求保存 Web 访问证据；实体之间再通过“攻击源发起事件、会话包含事件、原始请求关联事件”建立联系。

在概念模型层，系统进一步将安全事件度量拆分为威胁评分、风险等级和威胁等级三个层次。威胁评分是底层量化值，存放在原始请求表的威胁评分字段，用于表示单次 Web 请求触发规则后的累积分；评分越过阈值并升级为可疑事件后，系统在统一事件表中赋予该事件风险等级，用于标识单次事件的风险层次；再进一步，系统按来源 IP 或会话标识汇总历史行为，在会话追踪表中形成更稳定的威胁等级判断。该分层设计使系统能够由数值特征逐步过渡到实体研判。

基于上述实体及其关系，可构建系统数据库实体联系图，如图 4-9 所示。

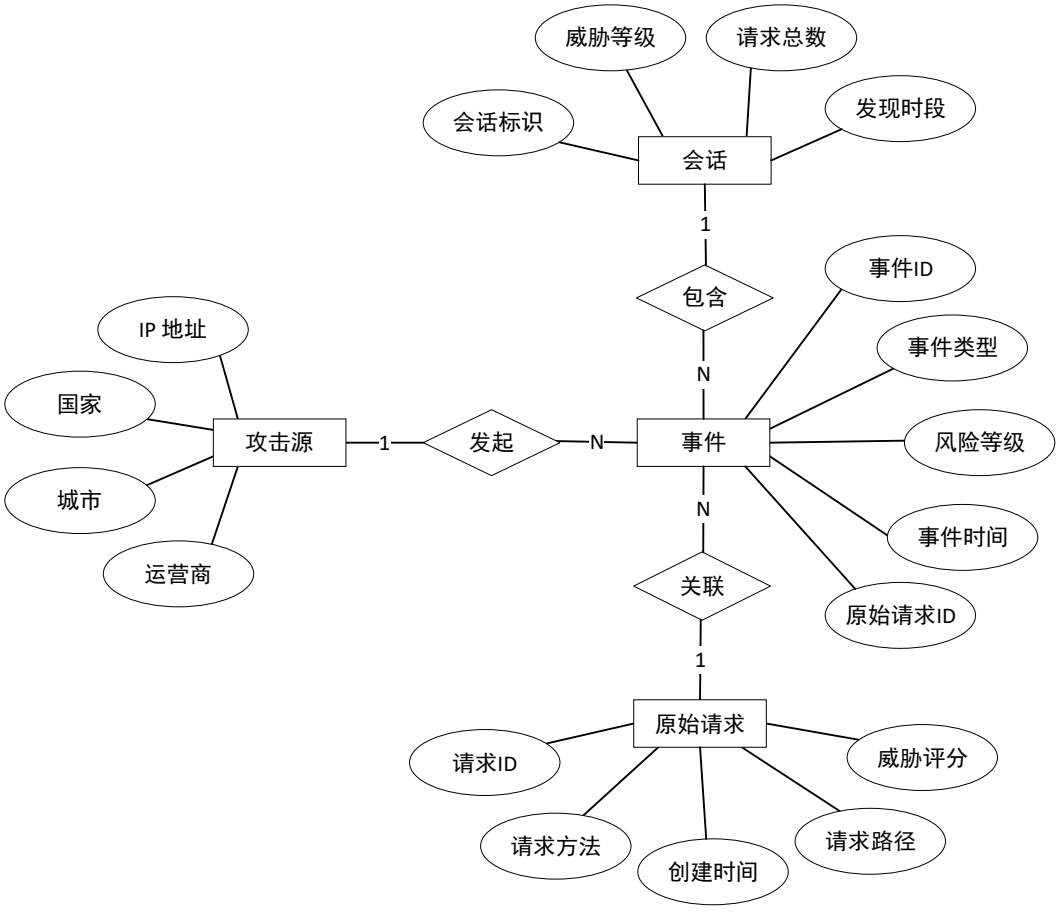


图 4-9 系统数据库实体联系图

后续能够从管理端继续回溯至上游证据，依赖的正是上述实体关系划分。事件实体位于中心位置，负责承接经治理后的结构化安全事件；原始请求实体保留 Web 侧访问证据，会话实体负责串联连续行为，攻击源实体补充来源属性与地理信息。按照这种组织方式，分析路径既可以从单条事件回溯到原始请求，也可以沿来源 IP 或会话标识继续查看前后活动，从而把“单次行为判断”扩展为“连续行为研判”。

4.4.2 逻辑结构设计

逻辑结构设计阶段把前文 E-R 结果进一步压实为可落库的关系模型。考虑到后续既要按事件回看，也要沿来源和会话继续追踪，数据库最终保留统一事件表、地理缓存表、原始请求表和会话追踪表四个关系模式。

统一事件（事件 ID，事件时间，事件来源，风险等级，事件类型，来源 IP，会话标识，标签数据，载荷摘要，原始引用，原始请求 ID，扩展信息）。

地理缓存（地理 ID，IP 地址，国家，省份，城市，纬度，经度，运营商，更新时间）。

原始请求（请求 ID，会话标识，来源 IP，请求方法，请求路径，查询参数，请求头，请求体，用户代理，引用来源，内容类型，内容长度，是否可疑，威胁评分，创建时间）。

会话追踪（记录 ID，会话标识，来源 IP，首次发现时间，最后发现时间，请求总数，路径序列，是否攻击者，威胁等级）。

四个关系模式在逻辑层承担的职责并不相同：统一事件用于存放经治理后的结构化事件，地理缓存用于复用 IP 解析结果，原始请求用于保留 Web 访问证据，会话追踪用于串联同源连续行为。按这种方式拆分后，联查既可以从单条事件回溯到原始请求，也可以沿来源 IP 或会话标识继续追踪会话活动。

联系转换阶段通过关联键建立实体关系。统一事件中的来源 IP、会话标识、原始请求 ID 分别关联地理缓存、会话追踪和原始请求，对应攻击源“发起”、会话“包含”、原始请求“关联”三类关系。在逻辑层，原始请求 ID 指向原始请求主键请求 ID，来源 IP 与会话标识则由关联规则和一致性约束共同维护。

关系模型按第三范式思路优化，核心实体关系满足去冗余和可追溯要求，同时保证各实体职责边界清晰、语义稳定。设计上不将原始请求字段直接并入统一事件实体，以降低冗余并保留上游证据链，避免单表过宽带来的维护复杂度；在此基础上，统一事件、原始请求、会话追踪和地理缓存四类实体共同支撑跨协议统计、关联检索与追溯分析。

4.4.3 物理结构设计

物理结构设计在上述关系模式基础上展开，结合运行与检索需求给出核心数据表字段定义和完整性约束。

为兼顾取证完整性和结构稳定性，统一事件表是统一事件模型在物理层的直接落点。SQL 注入等 Web 攻击采用“原始请求 + 统一事件”双表协同：原始请求表（raw_request）保存经脱敏和长度控制后的请求上下文，以及威胁评分、是否可疑等判定结果，统一事件表（event_log）通过事件类型、标签数据、载荷摘要和原始请求 ID（raw_request_id）记录结构化事件并回指原始请求记录。该设计无需频繁调整表结构即可支持规则扩展。在关联关系上，原始请求 ID 作为关联字段，实现统一事件到原始请求的单向追溯。统一事件日志表如表 4-1 所示。

表 4-1 统一事件日志表

字段名	字段描述	数据类型	完整性约束
id	事件 ID	INTEGER	主键，自增
timestamp	事件时间	DATETIME	非空
source	事件来源	VARCHAR(16)	非空
severity	风险等级	VARCHAR(16)	非空
event_type	事件类型	VARCHAR(64)	非空
src_ip	来源 IP	VARCHAR(45)	可空
session_id	会话 ID	VARCHAR(64)	可空
tags	标签数据	TEXT	可空
payload	载荷摘要	TEXT	可空
raw_ref	原始引用	VARCHAR(128)	可空
raw_request_id	原始请求 ID	INTEGER	关联字段，可空
extra	扩展信息	TEXT	可空

统一事件表不仅用于存储日志，还用于固定联查所需的公共字段。除事件 ID、事件时间和原始引用等基础字段外，其余业务字段主要回答三个问题：行为来自哪里、具体发生了什么、后续还能如何回查。事件来源用于区分 SSH 与 Web，事件类型用于标明认证失败、认证成功、命令审计、目录探测、注入尝试等行为；来源 IP 与会话标识负责串联同源行为和同次交互，风险等级用于支撑展示排序和告警分层；原始请求 ID 负责回指 Web 侧请求记录，标签数据用于保存扫描、弱口令、命令执行等可检索特征，载荷摘要用于在列表中保留关键信息，扩展信息则存放协议特有且暂不适合固化成列的细节。该设计不是把 SSH 和 Web 的原始字段机械拼成一张宽表，而是先统一公共字段，再把强关联证据留在原始请求表和扩展字段中，以兼顾统一检索、证据回溯和后续扩展。

地理缓存表用于来源地理信息的缓存、复用与快速查询，以降低重复解析开销并提升展示一致性，其字段定义与约束如表 4-2 所示。

表 4-2 地理位置缓存表

字段名	字段描述	数据类型	完整性约束
id	地理 ID	INTEGER	主键，自增
ip	IP 地址	VARCHAR(45)	非空，唯一键
country	国家	VARCHAR(64)	可空
province	省份	VARCHAR(64)	可空
city	城市	VARCHAR(64)	可空

续表 4-2

字段名	字段描述	数据类型	完整性约束
latitude	纬度	REAL	可空
longitude	经度	REAL	可空
isp	运营商	VARCHAR(128)	可空
update_time	更新时间	DATETIME	可空

来源地理信息单独设置，主要是因为这部分数据变化频率低、查询频率却高。该设计既能避免同一 IP 在大量事件中重复写入国家、省份、城市和运营商等字段，也便于地图展示和来源统计直接复用既有解析结果，从而降低重复解析开销并保持展示口径一致。

原始请求表用于保留 Web 侧访问证据并支撑风险复核，记录请求上下文与判定输入的主要内容，以便后续回放分析和规则迭代，其字段定义与约束如表 4-3 所示。

表 4-3 原始请求日志表

字段名	字段描述	数据类型	完整性约束
id	请求 ID	INTEGER	主键，自增
session_id	会话标识	VARCHAR(64)	可空
src_ip	来源 IP	VARCHAR(45)	非空
method	请求方法	VARCHAR(10)	可空
path	请求路径	VARCHAR(1024)	可空
query_string	查询参数	TEXT	可空
headers	请求头摘要	MEDIUMTEXT	可空
body	请求体摘要	MEDIUMTEXT	可空
user_agent	用户代理	VARCHAR(500)	可空
referer	引用来源	VARCHAR(500)	可空
content_type	内容类型	VARCHAR(100)	可空
content_length	内容长度	INTEGER	可空
is_suspicious	是否可疑	INTEGER	可空
threat_score	威胁评分	INTEGER	可空
create_time	创建时间	DATETIME	非空

原始请求表保留的是 Web 侧访问证据的主体内容，其职责边界与统一事件表并不相同。统一事件表更强调行为如何被归类和展示，原始请求表则侧重保留请求上下文及其评分依据。因此，这张表保留了方法、路径、查询参数、请求头、请求体、用户代理和内容

类型等主要上下文，用于支撑后续复核、样本回放和规则调整；是否可疑与威胁评分字段则用于记录该请求在是否升级为统一事件之前的原始判定结果。按此划分后，管理端列表无需直接展示大段原始参数，但在需要核对证据时仍能回到对应请求记录。

会话追踪表用于攻击链追踪与会话聚合，支撑同源行为串联、活跃度统计与威胁等级演进分析，其字段定义与约束如表 4-4 所示。

表 4-4 会话追踪表

字段名	字段描述	数据类型	完整性约束
id	记录 ID	INTEGER	主键，自增
session_id	会话标识	VARCHAR(64)	非空，唯一键
src_ip	来源 IP	VARCHAR(45)	非空
first_seen	首次发现时间	DATETIME	非空
last_seen	最后发现时间	DATETIME	非空
request_count	请求总数	INTEGER	可空
path_sequence	路径序列	TEXT	可空
is_attacker	是否攻击者	INTEGER	可空
threat_level	威胁等级	VARCHAR(16)	可空

会话追踪表关注的重点不是单条记录，而是“同一会话在一段时间内的连续行为”。首次发现时间和最后发现时间用于确定活跃区间，请求总数用于衡量行为密度，路径序列用于恢复主要访问轨迹，是否攻击者与威胁等级则用于给同一来源或同一会话形成更稳定的整体判断。统一事件表更适合观察单条行为，会话追踪表更适合观察连续过程，两者配合后，攻击链追踪才不会只剩下一串分散记录。

存储策略按运行阶段分开处理。联调和回归阶段使用轻量存储，便于快速部署；当并发规模增大或出现集中管理需求时，再切换到远程关系数据库。该安排先保证验证效率，同时保留扩展余量。

查询优化围绕高频检索路径展开：统一事件表围绕事件时间、事件来源、风险等级、事件类型、来源 IP 和原始请求关联字段建立索引，地理缓存表在 IP 地址上建立唯一索引，原始请求表围绕来源 IP、会话标识以及可疑标记、威胁评分、创建时间建立组合索引，会话追踪表围绕来源 IP、请求总数和最后发现时间建立索引。上述索引用于提升时间窗统计、可疑请求筛选和攻击链回溯效率。字段长度依据现网样本最大长度并预留安全余量；长载荷在入库前执行摘要提取与脱敏处理，以降低存储与查询压力。

在数据库物理结构之外，系统还围绕事件进入管理端之前的数据处理链路进行了设计。该链路依次覆盖事件采集、日志解析、数据清洗、地理富化和可视化输出，保证展示结果可以回溯到上游行为记录。态势感知数据处理流程图如图 4-10 所示。

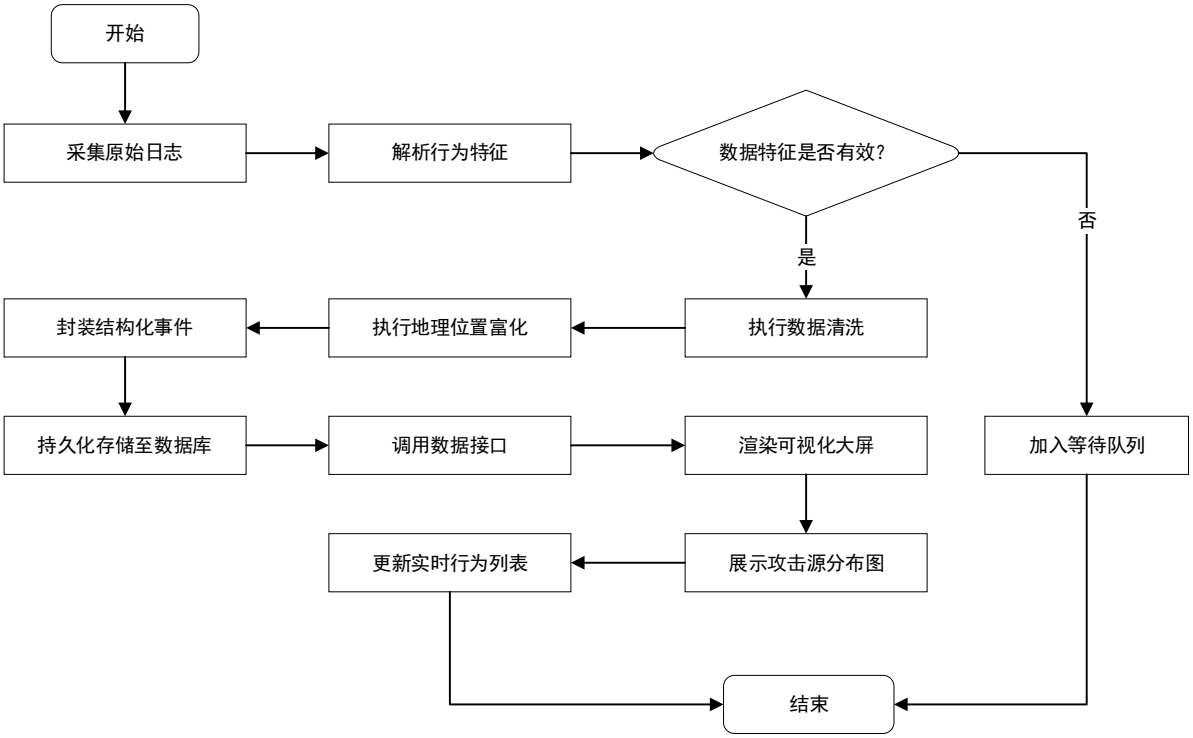


图 4-10 态势感知数据处理流程图

事件从进入系统到出现在管理端之间并不是一次性直通，而是要经过逐层整理。事件采集负责接收原始行为，日志解析与数据清洗负责将不同协议内容整理为统一口径，地理富化负责补充来源属性，可视化输出再把结构化结果转成适合观察的统计与明细。经过分段处理后，管理端展示的每一项结果都能够继续回指上游记录，避免出现“图表能看、证据找不到”的情况。

4.5 本章小结

本章完成了流程、结构、界面原型和数据库设计。分层架构与统一事件模型确定后，系统的模块划分、数据流转和管理端展示关系也随之明确，为后续实现与测试提供了设计依据。

5 系统实现

本章说明系统各模块的具体实现。SSH 侧采用线程隔离处理并发连接，Web 侧在请求链路提取关键特征，展示侧采用增量刷新以降低渲染开销。

5.1 系统环境配置

实际开发和联调都在 Windows 11 64 位主机上完成，后端运行环境使用 Python 3.11。第三方依赖集中放在 venv 中，前期默认接入 SQLite，编码和调试主要在 PyCharm 中进行；部署阶段不依赖固定操作系统，只要满足相应运行环境即可。

5.2 SSH 协议仿真功能实现

SSH 模块的处理分为认证处理和会话仿真两个阶段，具体流程如图 5-1 所示。

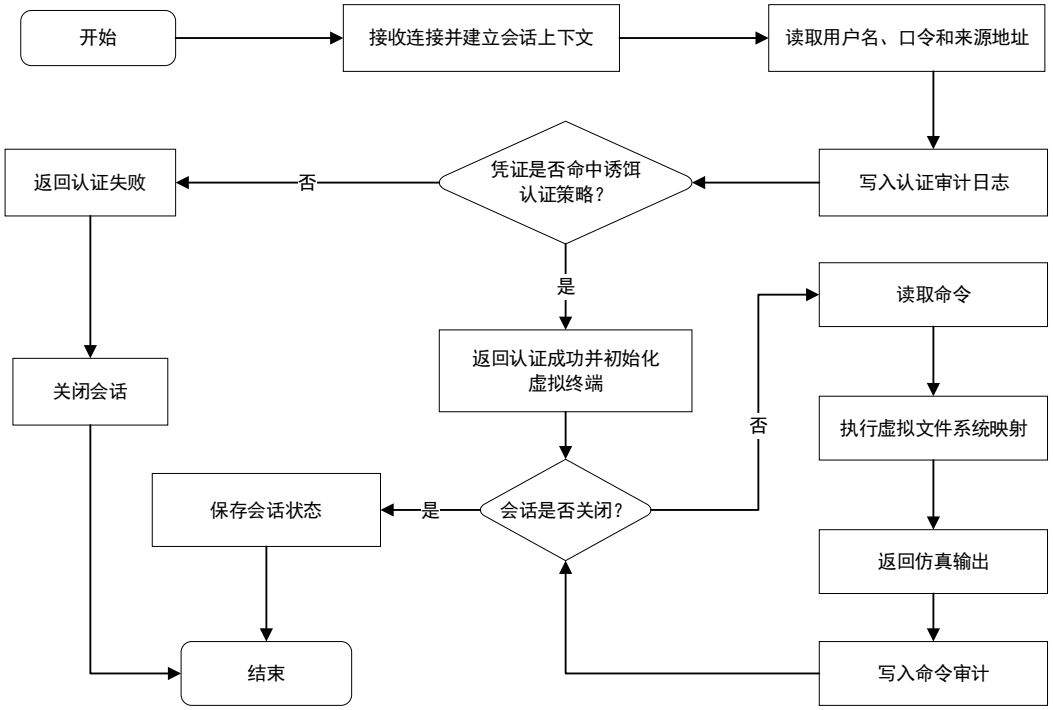


图 5-1 SSH 认证与命令处理程序流程图

该模块既不采用只返回固定拒绝语句的低交互方式，也不直接暴露真实系统环境，而是在认证、终端提示符、目录切换和常见命令回显等关键环节提供有限但连续的交互。该设计在样本深度和运行风险之间取得了平衡：既保留后续命令交互，又能在受控边界内记录操作过程。

SSH 端没有沿用默认认证流程，而是在 ServerInterface 回调中接管口令校验。收到登录尝试后，程序先施加短时随机等待，再记录用户名、口令样本、来源地址和连接标识，随后依据预设凭证集返回认证结果。登录结果分别记录为认证失败事件和认证成功事件；进入交互阶段后，命令审计记录仍沿用同一连接标识继续串联。

为避免高频爆破和重复输入造成无效刷库，SSH 侧并未对认证事件做统一归并，而是保留每次认证成功与失败记录，以便后续还原爆破过程；进入交互阶段后，命令审计仅对连续相同命令做去重，可疑命令事件则按相同风险签名抑制重复写入。这样既能保留认证轨迹，又能减少无意义的重复命令噪声。

会话仿真阶段在认证成功后启动，系统按连接级隔离处理并发请求。每个客户端在独立执行上下文中维护命令缓冲、光标状态和历史记录；交互循环持续读取字节流输入，再分发到虚拟命令执行模块进行解析与回显。命令执行不会触达宿主机真实文件系统，而是基于内存级虚拟目录和文件映射返回仿真结果，例如目录切换、文件读取和进程信息查询均在会话态内完成。

此外，模块在接入前对总并发连接数和单源连接数进行限制，防止高频探测导致服务资源被耗尽；文件传输链路对上传体积设置上限并记录样本事件，形成“认证拦截—命令审计—样本留存”的闭环诱捕路径。

5.3 Web 漏洞诱捕功能实现

Web 模块主要由入口约束、风险识别和诱饵响应三个环节构成，其整体处理流程如图 5-2 所示。

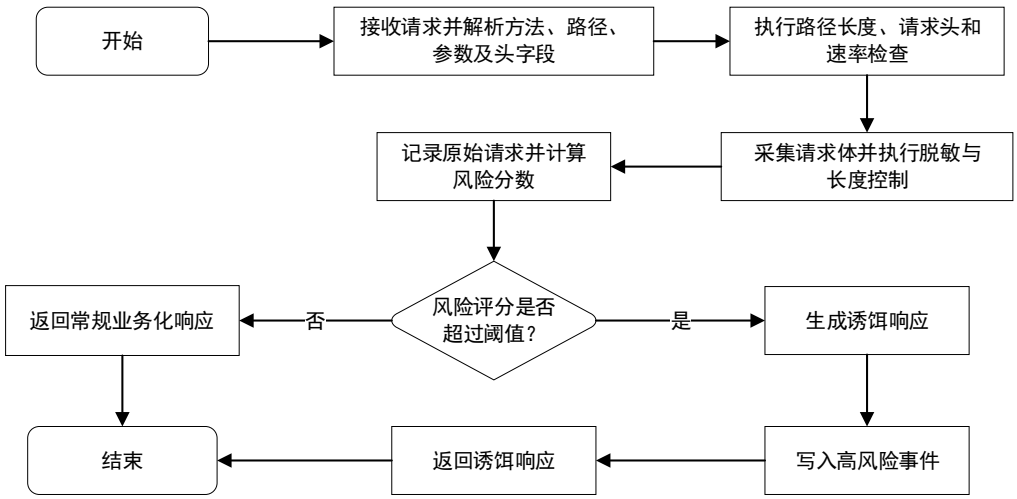


图 5-2 Web 请求分析与诱捕程序流程图

在风险识别环节，系统不采用命中单条规则即直接告警的处理方式，而是采用“**多特征加权 + 双阈值分层**”机制，对单次请求进行量化评分，并将展示链路与告警链路拆开

处理。这是因为单个信号往往不足以直接证明攻击成立，例如一次后台路径访问可能只是探测，一次特殊字符命中也可能来自异常输入；只有把多种信号放到同一次请求和同一时间窗口中综合判断，风险分层才更稳定。

请求进入后，系统先检查路径长度、查询串长度和请求头规模。超限请求在拦截前仍保留审计与评分结果；通过约束的请求再写入原始请求表，并进入风险评分链路。该设计在控制资源消耗的同时保留了主要请求证据。

SQL 注入识别采用“请求归一化 + 特征匹配 + 风险评分”的三阶段处理机制。首先，针对攻击载荷可能存在的 URL 编码绕过问题，系统会对查询参数与请求体执行最多两轮 URL 解码，并统一转换为小写文本用于稳定匹配。其次，在特征匹配阶段，系统对常见 SQL 注入模式进行规则检测，特征覆盖 ’ or、union select、information_schema、sleep() 等典型语义信号，并结合上下文组合语义降低单符号误判。

多特征加权是指，系统不采用命中即判定的二值方式，而是对路径、请求头、参数和请求体中的风险信号按权重累计评分，其中明显利用信号权重更高，弱异常仅作为辅助依据参与加分。双阈值分层是指，累计得分达到 20 分记为可疑事件，达到 40 分记为高危告警；对带有头部异常且单次未越过可疑阈值的低分请求，再引入 5 分钟会话累积抬升机制，以降低短时漏报。这样处理后，系统既能保留持续观察所需的异常记录，也能将真正需要联动提示的高风险请求区分出来。核心权重项如表 5-1 所示。

表 5-1 Web 漏洞风险评分权重表

特征项	权重（分）	识别逻辑模块
路径探测（敏感/扫描/LFI）	+15 至 22	路径分析模块
请求头检测（指纹+异常）	+2 至 22	请求头分析模块
参数字符异常（特殊字符+空字节/换行）	+12 至 20	载荷分析模块
SQL 注入语义（组合语义+标记命中）	+18 至 25	载荷分析模块
命令执行/WebShell 特征	+18 至 28	参数行为分析模块
会话累积抬升（5 分钟）	+4 至 8	会话行为分析模块
异常请求体（反序列化/表达式）	+20 至 25	载荷分析模块

区间分值的设置，并非为了把每类风险写成固定结论，而是为了反映同类特征在不同命中情形下的确定性差异。以路径探测为例，普通后台试探与明确的目录穿越虽然同属探测行为，但后者风险更高；SQL 注入语义也是如此，弱特征与典型组合语义的可信度存在差异，因此需要分档赋分。相关权重主要参考 OWASP Top 10 涵盖的常见 Web 风险类型，并结合本系统“双阈值分层”机制，在功能测试、攻击回放和外网运行过程中根据误报、漏报情况反复调整。整体上，探测类信号分值较低，利用类信号分值较高，目的不是凭单一特征直接下结论，而是让多种迹象在同一次请求中逐步累积。按照当前规则，累计达到

20 分记为可疑事件，便于后续展示与回看；累计达到 40 分时触发高危告警。例如，对后台敏感路径的访问命中 15 分，同时参数中出现较明确的 SQL 注入语义再加 25 分，请求即可达到 40 分并进入高危告警链路。会话累积抬升仅针对带有头部异常且短时间内反复出现的低分请求，用于弥补单次弱信号不易越过阈值的情况。

对于目录扫描和后台探测，系统在路由层预置了 /backup/、/.env、/.git/config 和 /admin/<path:subpath> 等高风险入口。请求命中后先记录探测事件，再结合访问场景返回伪响应，并叠加随机延时，以降低脚本快速识别蜜罐的概率。后台仅保留少量受控后台流程，相关操作仍限制在仿真态和隔离态范围内，不触达真实宿主机与真实业务配置。

在落库与联动阶段，请求通过预检后先写入原始请求表保存证据，再进入评分判断；达到可疑阈值后写入统一事件表，达到高危阈值后触发企业微信告警。高危告警并不是系统唯一的处置动作。在此之前，请求已经经过路径长度、查询长度、请求头规模等约束，并叠加限流、随机延时、伪装响应和仿真隔离等处理。因此，评分机制主要承担分层研判与告警联动的职责，而不是把阈值直接作为封禁开关。

5.4 攻击日志采集与解析实现

攻击日志采集与解析的核心在于统一事件模型。这里的“统一”并非把两类日志机械拼接，而是从管理端联查需求倒推字段设计：凡是 SSH 与 Web 都需要参与统计、检索和追踪的内容，如来源地址、事件类型、风险等级、会话标识和摘要信息，统一进入事件层；只有 Web 独有且需要继续留存的请求上下文保存在原始请求表，SSH 侧则通过连接标识把认证记录、命令审计和高危命令事件串联起来。对应的事件来源、事件类型、来源 IP、会话标识、标签数据、载荷摘要和原始请求 ID 等公共字段已在表 4-1 中给出。在这一设计下，管理端查询时先看统一事件，再按原始请求 ID 或会话标识回指底层证据，不必在前端反复做字段对齐和格式清洗。

两条落库路径在实现上并不相同。通过请求预检且未被资源约束拦截的 Web 请求，无论是否命中风险规则，都会先写入原始请求表，并同步更新会话追踪表；仅当风险分达到 20 分及以上时，才升级写入统一事件表。对超限请求，系统则保留审计与评分结果，用于后续回看与分析。SSH 侧的认证成功、认证失败、命令审计以及高危命令事件则直接写入统一事件表，并通过 session_id 维持同一连接内的前后关联。写入统一事件前，系统会统一标准化事件来源、风险等级和事件类型，再对摘要与扩展信息执行字段整理和长度控制，以保证检索与展示口径一致。高频字段建立索引后，检索和聚合效率能够稳定满足大屏查询需求。

地理解析采用两级缓存策略。系统先查询地理缓存表，未命中时再调用外部 IP 解析并回填，同时配合带过期时间的内存缓存，减少地图场景下的重复解析。为保证高频写入与查询的稳定性，SQLite 侧启用预写日志机制和超时等待机制，并结合锁冲突重试与面

向时间、类型和可疑请求筛选路径的索引优化，保障高频查询场景的响应稳定性。

5.5 态势感知大屏展示实现

态势感知大屏采用定时拉取配合增量刷新的方式。后端由 Flask 提供结构化 JSON 接口，覆盖事件列表、统计汇总、地理来源、可疑请求和会话分析等查询能力；各接口统一返回状态码、提示信息和数据内容，并在访问控制层拦截未登录请求，保证管理面与诱捕面隔离。前端按“近 24 小时、近 7 天、近 30 天”三个时间窗口组织数据展示，避免不同统计口径混用。

前端页面加载完成后先初始化 ECharts，再启动分频轮询。时间信息按秒更新，主数据模块按设定周期刷新，服务器信息每 15 秒更新一次，避免整页重绘造成闪烁。数据获取采用并发请求与分层更新相结合的方式：系统先拉取事件统计、地理来源和日志主数据，再异步补齐可疑请求与会话分析，使切换时间窗口时核心指标能够优先呈现。为避免旧请求回包覆盖新数据，前端增加窗口一致性校验，仅接受当前窗口对应的返回结果。

当请求短时异常时，系统优先使用同窗口最近一次成功数据；若“近 24 小时”出现短时空窗，则回退展示最近历史窗口数据，并用状态标识说明当前为历史数据。日志模块按日志编号去重后增量插入，只更新变化行，减少重绘闪烁。管理员可通过大屏实时查看风险分布、来源排行与事件明细，并在事件明细区继续追踪原始请求和会话标识，运行界面如图 5-3 所示。



图 5-3 态势感知大屏运行界面图

在可疑请求详情窗口中，管理端进一步展示攻击源 IP、攻击源位置、判定依据、请求方法、URL、请求头和请求体摘要等内容。该窗口用于把大屏中的可疑事件继续下钻到具体请求记录，便于核对触发规则与请求上下文。可疑请求详情如图 5-4 所示。



图 5-4 可疑请求详情图

攻击链详情窗口按照来源 IP 聚合同源连续行为，并展示入口类型、攻击源位置、行为次数、威胁等级以及认证失败、认证成功和命令审计等事件顺序。通过该窗口，管理员可以从单个攻击源继续观察其连续活动过程，而不是只查看孤立的日志明细。攻击链详情如图 5-5 所示。



图 5-5 攻击链详情图

在展示层，管理端未将 SSH 与 Web 分成彼此独立的两套观察面板，而是围绕统一事件模型组织统计总览、来源分布、事件明细和攻击链追踪。管理员看到的不是彼此割裂的两类日志，而是能够继续回指原始请求、会话标识和来源聚合结果的完整链路，统一事件模型在展示层的组织作用也因此得到了体现。

5.6 本章小结

本章给出了环境配置、协议仿真、Web 诱捕、日志治理和可视化展示的实现过程。至此，SSH 诱捕、Web 诱捕与管理端已经能够按统一事件模型协同运行，并完成从采集、入库到展示的整条处理链路。

6 系统测试

6.1 系统测试目的

本章测试以可用性、稳定性和可追溯性为主线，测试范围覆盖功能、性能与安全三个维度。功能层重点核对 SSH 交互、Web 诱捕、统一日志落库以及大屏联动是否按预期连通。性能层主要观察核心接口在持续调用与并发访问条件下的响应时延和返回情况。安全层则检查异常输入识别、权限控制与告警链路是否完整打通且可复现。

系统测试以黑盒验证为主，并结合过程观测。测试拓扑采用“外网流量接入 + 管理端回查”的分离方式：SSH 仿真端与 Web 诱捕端持续对外开放，攻击流量通过 SSH 客户端和浏览器脚本进入系统，管理端在受控侧完成统计、联查与接口验证。场景覆盖端口扫描、口令爆破、目录探测和参数攻击。环境搭建遵循最小影响原则，测试主机与目标服务分离，相关进程保持受控，避免捕获过程影响宿主稳定性。测试主机为 8 核 CPU、16GB 内存、固态硬盘，平台为 Windows 11 64 位，运行环境为 Python 3.11 + venv；实验阶段采用 SQLite，扩展部署阶段采用 MySQL。围绕协议仿真、风险识别与三端协同链路，共设计 17 项综合用例。

6.2 系统测试过程

所有用例按“测试输入 - 预期行为 - 实际结果”的方式记录，并结合外网连续运行观测结果交叉验证。连续运行期间，统一事件表累计记录事件 85082 条，其中 SSH 事件 84273 条、Web 事件 809 条。Web 注入专项另使用脚本生成 100 个攻击样本和 30 个正常样本，前者由多组 SQL 注入模板循环构造，后者采用普通查询参数请求。上述数据作为测试分析参照，具体结果如表 6-1 所示。

表 6-1 系统综合测试用例表

测试功能	测试内容	预期结果	实际结果	是否通过
SSH 凭证认证功能	使用预置有效账号与口令发起 SSH 登录	登录成功并进入仿真终端，记录认证事件	登录成功并进入仿真终端，记录认证事件	通过
SSH 凭证认证功能	使用不存在用户名与任意口令尝试登录	拒绝登录并写入认证失败事件	拒绝登录并写入认证失败事件	通过
SSH 凭证认证功能	提交空口令登录请求	拒绝登录并提示认证失败	拒绝登录并提示认证失败	通过

续表 6-1

测试功能	测试内容	预期结果	实际结果	是否通过
SSH 会话交互功能	登录后连续执行 mkdir doc、cd doc、pwd	连续执行后路径状态正确	连续执行后路径状态正确	通过
SSH 文件仿真功能	在仿真终端执行 cat /etc/passwd	返回脱敏仿真内容	返回脱敏仿真内容	通过
SSH 会话管理功能	登录后强制关闭 SSH 客户端连接	服务端释放会话并保留审计记录	服务端释放会话并保留审计记录	通过
Web 页面响应功能	访问 /index 与 /news/list 常规页面	返回常规页面且不触发高危告警	返回常规页面且不触发高危告警	通过
Web 路径诱捕功能	请求 /admin/config.php 等敏感路径	受控响应并新增目录探测事件	受控响应并新增目录探测事件	通过
Web 注入检测功能	在参数中提交 ’ or 1=1 -- 载荷	识别注入探测并关联原始请求	识别注入探测并关联原始请求	通过
Web 恶意载荷检测功能	在参数中提交 <script>alert(1)</script> 载荷	标记可疑请求并保留参数摘要	标记可疑请求并保留参数摘要	通过
Web 方法校验功能	对受控路径发送异常 HTTP 方法请求	返回受控响应并记录异常方法事件	返回受控响应并记录异常方法事件	通过
Web 诱饵资源功能	访问 /old/site-2024.tar.gz 诱饵资源	返回诱饵响应并记录文件探测事件	返回诱饵响应并记录文件探测事件	通过
管理端事件联查功能	在管理端结合来源 IP 与会话标识执行联查	可联查完整事件链路	可联查完整事件链路	通过
管理端刷新容错功能	在网络抖动条件下持续刷新态势页面	显示加载提示，网络恢复后数据继续刷新	显示加载提示，网络恢复后数据继续刷新	通过
管理端权限控制功能	未登录状态直接调用管理端接口	返回拒绝响应且不泄露管理数据	返回拒绝响应且不泄露管理数据	通过
接口性能测试	连续调用管理端核心接口 500 次	平均响应时间 < 1 s，接口可稳定返回	接口稳定返回，满足秒级响应	通过
并发压测验证	模拟 100 并发请求访问 Web 诱捕路由	系统稳定运行并返回可控响应	并发稳定，响应可控并形成审计记录	通过

测试期间，系统对高频 SSH 异常认证请求进行受控拦截，并记录攻击源 IP、错误凭

证尝试次数等审计字段，测试结果如图 6-1 所示。

```
May 13 22:27:03 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:27:03] [INFO] [ssh] [失败] SSH认证失败 155.212.168.71: root/Admin2023!
May 13 22:27:13 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:27:13] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/windows
May 13 22:27:46 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:27:46] [INFO] [ssh] [失败] SSH认证失败 195.178.110.4: root/!QAZxsw2
May 13 22:27:49 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:27:49] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/zxcde456
May 13 22:28:25 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:28:25] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/utstar
May 13 22:28:44 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:28:44] [INFO] [ssh] [失败] SSH认证失败 155.212.168.71: root/knight
May 13 22:29:03 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:29:03] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/admin001
May 13 22:29:36 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:29:36] [INFO] [ssh] [失败] SSH认证失败 195.178.110.4: root/12
May 13 22:29:42 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:29:42] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/zhongguochina.+
May 13 22:29:58 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:29:58] [INFO] [ssh] [失败] SSH认证失败 156.238.224.50: hdfs/hdfs
May 13 22:30:18 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:30:18] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/1qazwsx2
May 13 22:30:22 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:30:22] [INFO] [ssh] [失败] SSH认证失败 155.212.168.71: root/mac
May 13 22:30:55 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:30:55] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: patricia/patricia
May 13 22:31:25 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:31:25] [INFO] [ssh] [失败] SSH认证失败 195.178.110.4: root/qwerty
May 13 22:31:30 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:31:30] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/zxc123!
May 13 22:31:58 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:31:58] [INFO] [ssh] [失败] SSH认证失败 155.212.168.71: root/gituser
May 13 22:32:05 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:32:05] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: root/zxcvbnmm
May 13 22:32:41 cn-proxy bash[3371597]: [SSH] [2026-05-13 22:32:41] [INFO] [ssh] [失败] SSH认证失败 167.99.72.161: worker/worker
```

图 6-1 SSH 异常认证测试结果图

该测试重点核对两点：一是异常认证请求能够被服务端稳定识别并拒绝，二是拒绝动作发生后，来源地址、尝试账号和认证结果等审计字段仍然能够完整保留。因此，该图反映的并非单纯的登录失败，而是认证拦截和事件留痕在同一次交互中同时成立。

在 Web 恶意请求场景下，系统按评分阈值执行拦截、落库和告警联动，结果如图 6-2 所示。

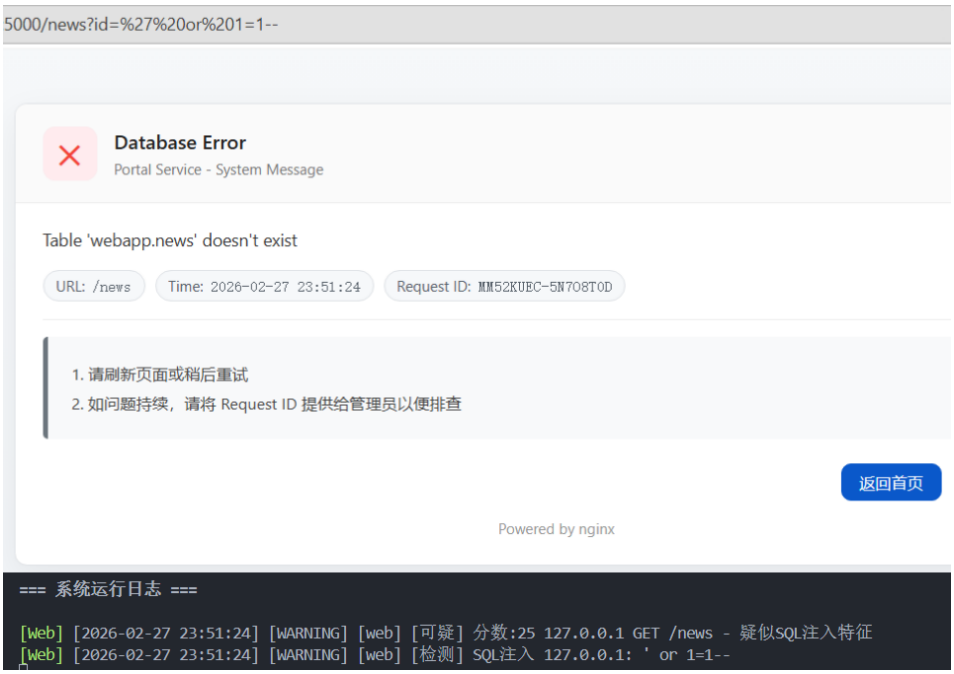


图 6-2 Web 异常请求拦截测试结果图

Web 侧测试覆盖原始请求捕获、风险评分和统一事件升级三个环节。测试重点不只是请求被识别为异常，还包括评分结果是否能够推动后续分层处置，即原始请求是否入库、可疑事件是否生成、高危样本是否继续联动告警。该图可用于检验“双阈值分层”在实际链路中是否真正发挥作用，而不是停留在规则说明层面。

6.3 系统测试结论

17 项功能用例全部通过，说明 SSH 与 Web 的核心诱捕流程、统一事件记录以及管理端联查链路均已打通，统一事件模型也已在实现层完成落地。SSH 登录成功、认证失败、命令审计以及 Web 路径探测、参数攻击等结果均可写入统一事件表，并继续通过来源 IP、会话标识和原始请求 ID 回指对应证据，因此管理端联查具备了实际应用价值。压测结果进一步反映了系统在实验环境中的持续运行能力：在上述“外网流量接入 + 管理端回查”的测试拓扑下，管理端核心统计接口连续调用 500 次时平均响应时间为 134.9 ms，95% 请求在 168.71 ms 内返回，成功率为 100.0%；Web 诱捕路由在 100 并发压测下完成率为 100%，返回结果均为非 5xx 可控响应，平均响应时间为 350.7 ms，95% 请求在 787.23 ms 内返回。由此可见，在持续采集、落库与展示并发存在的条件下，系统仍能保持稳定响应，并非仅适用于离线演示场景。

在该轮 Web 注入专项测试中，100 个攻击样本覆盖布尔判断、联合查询、报错注入、时间延迟和注释绕过等常见形式，其中 82 个被识别并进入落库链路，捕获率为 82.0%；30 个正常样本未被误判，误报率为 0.0%。结果表明，现有评分规则对常见 SQL 注入探测已有较稳定的识别效果，多特征加权与双阈值机制能够把明显攻击样本抬升到展示层或告警层，同时避免把正常请求大面积卷入告警通道。未命中的样本主要反映出现有规则对复杂混淆、弱特征组合和变形载荷的覆盖还不充分，后续仍需围绕规则细化、样本补充和并发场景继续验证。

6.4 本章小结

本章从测试方案、执行过程和结果分析三个方面对系统进行了验证。根据现有测试证据，系统在功能正确性、运行稳定性和威胁捕获效果方面已达到毕业设计的基本要求；后续若需进一步完善，重点仍在于扩大样本规模并细化评分规则。

结论

本文围绕网络安全实验场景下“如何持续留住攻击行为”和“如何把异构日志放到同一口径下分析”两个实际问题，设计并实现了一套面向网络威胁捕获的 Web 与 SSH 联合诱捕系统。全文工作的重点在于两项相互支撑的设计：一方面，统一事件模型把 SSH 登录试探、命令交互以及 Web 路径探测、参数攻击等行为纳入同一处理链路，使统计、追踪和证据回查能够围绕同一事件链路展开；另一方面，多特征加权与双阈值分层机制把需要保留观察的可疑行为与需要即时提示的高危行为区分开来，降低了低风险噪声对告警通道的挤占。与单协议分别部署、分散处理的方式相比，本文方案更便于开展跨协议联查，也更符合实验场景下对部署复杂度和运行边界的要求。

从系统运行结果来看，系统已经能够对暴力破解、目录扫描、命令探测和常见注入探测等行为进行持续记录，并通过管理端按时间窗口、来源地址和会话标识开展联动分析。连续外网运行期间共记录事件 85082 条，表明系统具备持续运行并稳定记录事件的能力。测试结果显示，17 项功能用例全部通过，管理端核心接口连续调用 500 次的成功率为 100.0%，Web 注入专项回放捕获率为 82.0%。这些结果表明，本文方案在实验环境下能够满足可用性、稳定性和可追溯性的设计要求。

但当前系统仍存在一定局限。首先，Web 侧对复杂混淆载荷的识别能力还有提升空间，部分复杂变体仍可能漏报；其次，现阶段测试样本规模和部署规模有限，结论主要建立在单节点、单时段运行结果基础上，对更高并发、更复杂拓扑和更长周期场景的验证还不充分。后续工作将重点放在三个方面：一是继续丰富攻击样本并优化评分规则，提升复杂场景下的识别效果；二是验证多节点协同采集与分层存储方案，增强系统扩展能力；三是结合更长期的外网运行数据，进一步提高系统在真实环境中的稳定性与分析价值。

参考文献

[1]中华人民共和国国务院.关键信息基础设施安全保护条例[EB/OL].(2021-08-17)[2026-03-26].https://www.gov.cn/zhengce/content/2021-08/17/content_5631671.htm.

[2]贵州省通信管理局.工业互联网网络安全情况通报（2024 年第 10 期）[R/OL].(2024-11-15)[2026-03-08].https://gzca.miit.gov.cn/xwdt/gg/art/2024/art_dd971c18b6cb46029c3f3347026678ef.html.

[3]OWASP FOUNDATION.OWASP Top 10:2025[EB/OL].[2026-03-08].https://owasp.org/Top10/2025/.

[4]Shahid J, Hameed M K, Javed I T, et al. A Comparative Study of Web Application Security Parameters: Current Trends and Future Directions[J]. Applied Sciences, 2022, 12(8):4077.DOI:10.3390/app12084077.

[5]Ilg N, Duplys P, Sisejkovic D, et al. A survey of contemporary open-source honeypots, frameworks, and tools[J]. Journal of Network and Computer Applications, 2023, 220:103737.DOI:10.1016/j.jnca.2023.103737.

[6]COWRIE PROJECT.Cowrie Documentation[EB/OL].[2026-03-26].https://docs.cowrie.org/en/latest/.

[7]MUSHORG.Glastopf Project Repository[EB/OL].[2026-03-26].https://github.com/mushorg/glastopf.

[8]赵仲力,李庆.基于 Cowrie 蜜罐系统的机房网络主动诱捕技术应用研究[J].无线互联科技,2024,21(15):117-119.

[9]杨诺,郭开阔,陈云.面向播出系统和内网防护的终端攻击诱捕系统的设计与实现[J].现代电视技术,2023(06):144-147.

[10]Bao T, Tambe M, Wang C, eds. Cyber Deception: Techniques, Strategies, and Human Aspects[M]. Cham: Springer, 2023.DOI:10.1007/978-3-031-16613-6.

[11]PYTHON SOFTWARE FOUNDATION.Python 3.11.15 Documentation[EB/OL].[2026-05-10].https://docs.python.org/3.11/.

[12]PALLETS PROJECTS.Flask Documentation (3.1.x)[EB/OL].[2026-05-10].https://flask.palletsprojects.com/en/stable/.

[13]PARAMIKO AUTHORS.Paramiko Documentation[EB/OL].[2026-03-26].https://docs.paramiko.org/.

[14]SQLITE DEVELOPMENT TEAM.About SQLite[EB/OL].[2026-05-10].https://www.sqlite.org/about.html.

[15]APACHE ECHARTS COMMUNITY.Handbook - Apache ECharts[EB/OL].[2026-05-10].https://echarts.apache.org/handbook/en/.

[16]国家互联网信息办公室.国家网络安全事件报告管理办法[Z/OL].(2025-09-15)[2026-03-08].https://www.cac.gov.cn/2025-09/15/c_1759583017717009.htm.

致谢

本论文的完成离不开指导教师林翔老师的悉心指导。林老师在课题选题、系统设计和论文修改阶段提供了持续帮助，在此致以诚挚谢意。

感谢雷新锋老师、钱庭霖老师和林玲老师在论文撰写与完善过程中给予的宝贵建议，感谢同学们在系统测试阶段提供支持，感谢家人在学习期间给予保障与鼓励。